Coral-Chords documentation

Linus Tibert

August 4, 2025

Contents

1 Used abbreviations

Although some of those might seem obvious, I'll give a complete overview of special abbreviations used in the document for those who are not familiar with any of them.

CCh	Coral-Chords
UG	Ultimate Guitar
BE	Backend (coral-chords-backend library)
FE	Frontend (coral-chords binary application)
UI	User-interface
URL	Uniform resource locator (web-link)
API	Application programming interface
ID/UID	(unique) identifier

2 Introduction

This document is supposed to be a useful guide for those who want to understand how the code of CCh works. It is especially designed to be an explanation of how each component of the program is working, however, some detailed parts (such as specific functions()) are not mentioned here, because they are easier to understand (if you know what the whole section of the code is for), when looking at the actual code.

To make this document more accessible for everyone, it is divided into many small sections, each with a clear subject.

3 Project structure

The project is divided into two major parts: the BA (coral-chords-backend library), and the main program (coral-chords binary; the FE).

The FE is the part of the program which interacts directly with the user and which is also creating the UI.

The BA is where the magic happens. Its purpose is to do anything not directly involving the user. Thus, it fulfils tasks involving networking, data extraction or direct interaction with the (file) system.

3.1 Backend

Because the BA still has many tasks, it is subdivided into three sub-modules: `data_preparation`, `system_access` and `types_and_constants`.

3.1.1 Data preparation module

This module is mainly used to retrieve data from UG and process it into useable data. It has the public function `get_song_data_from_url()`, which calls a stack of other functions to get the data from a URL as a `SongData` struct with as much metadata as possible.

3.1.2 System access module

Every direct interaction with the user's system is managed by this module. It is mainly used to read and write files and for detection of the currently running song on Spotify.

3.1.3 Types and constants module

All globally used types and constants are defined here.

This module mainly exists to keep the code clear. Having all custom types in one module makes it easier manage them across all parts of the program.

3.2 Frontend

4 Functionality details

4.1 Getting the data from UG

Ultimate Guitar does not have an API available for developers to build their code on. Thus, the best way to get UG's data is to get their source HTML and run a couple of Regex patterns on it to get raw data.

This is called "web scraping" and is exactly what CCh does. This way CCh gets a very limited kind of API to get machine processable data from UG. In the following sections I will elucidate the way CCh scrapes the data from raw HTML on UG.

4.1.1 Legal aspects of web scraping

First, a tiny disclaimer: Web scraping is not illegal, and in this case it is legal too.

The important thing in this case is that CCh is not downloading/scraping any restricted or confidential data. All it really does is downloading the same contents your web browser is downloading as soon as you look at a tab at UG.

Besides those aspects, CCh does not contain any scraped data; with publishing this project, I only supply people with an easy method of getting UG's tabs in a custom UI without re-publishing any of the copyrighted materials of UG.

To read more about this topic, you can visit my source of those facts in the footnote.[1]

4.1.2 Scraping UG tabs

Everything explained in this section is happening in the `data_preparation` module and is triggered by the `get_song_data_from_url()` function.

What CCh is doing can essentially be described as three major steps:

1. Download data from UG

2. Extract valuable information

3. Output data in a custom format

In the first step, a simple function is triggered which returns the raw HTML of the selected UG page. To make sure the downloaded data is valid for data extraction, the program simply checks for a few strings within the HTML; if certain ones are present and certain ones are not present, the page is valid.

[1] https://blog.apify.com/is-web-scraping-legal/

4.2 Storing the data

4.3 Interaction with Spotify

5 Core code components

This section will give you a brief overview of the most important self defined parts of the code. All of those components are defined in the `types_and_constants` module for global usage.

5.1 Constants

`END_OF_CHORDS_DELIM` is a short but unique piece of code from the UG source HTML, which is used to mark the end of the section containing the chords and lyrics.

`START_OF_CHORDS_DELIM` the same as `END_OF_CHORDS_DELIM`, but used to mark the beginning of useable data. Note that the metadata is not included between the delimiters; it is above the start delimiter.

`HTML_BLACKLIST` is an array of strings which identify certain types of UG pages which are not downloadable; video pages for example.

`DETAIL_REGEX` is the Regex expression used to get detailed metadata from the raw HTML of a supported UG page. Each capturing group holds one property.

`TYPE_REGEX` is used to extract the type of supported page. (I.e. guitar chords, tabs, ukulele chords, bass tabs or drum tabs) This data is in the second capturing group. The first group extracts the artist's name, which is not used in the code yet.

5.2 Structs

`DataLine` is used to store a single line of extracted data together with the type of data stored on the line. Those lines are used in the `SongData` struct.

`SongData` is used to store a single song with all its collected data. The `lines` are the same lines which are written and loaded in local files.

`BasicSongData` holds the basic metadata every song has. It consists of the song title, artist, and the Spotify song UID.

`SongMetadata` is used to store optional metadata about a song. It is able to hold the capo position, tonality, tuning name and tuning details. Some of this data is not available on every supported page; in such cases, the unavailable data will an empty string. If no metadata is available at all, `SongMetadata::default()` will be used as a metadata struct.

6 Tests

The library contains a bunch of tests to verify the code is still compatible with the currently available chords on UG.

The tests are named with what they are testing for. For example, the `type_detection()` test compares the results of detecting the types of a bunch of URLs using `get_type()` with a pre-defined type.

All tests are designed to work with as many types of pages as possible. This doesn't only include types like "Bass", "Guitar", etc., but also special pages of the same type (e.g., pages with more or less metadata than others).

If you extend the code in any way, make sure, all tests are passing before you submit a pull request. If you add support for any other type of page than currently supported, make sure to extend all tests appropriately.